6 EVALUATING A SOLUTION

OBJECTIVES

- Discuss key quality measures of a solution's effectiveness, including:
  - correctness using empirical means;
  - efficiency by examining time and space complexities;
  - elegance, specifically the maximisation of effectiveness and simplicity;
  - usability according to industry-standard measures.
- Explain the importance of trade-offs with regard to quality measures.

SOLUTION EVALUATION

You did it. You analysed the problem, broke it down into pieces, came up with a solution design, which you then implemented, tested and debugged. You have something that functions. So, you're finished, right?

Well, not quite.

The work is not over just because you have a solution. Before you can really finish, you have to make sure you've produced a **good** solution. You must evaluate its quality.

There are many aspects to quality. Evaluating a solution involves asking several basic questions, each addressing a specific aspect. Important questions about the solution include:

- Is it correct? Does it actually solve the problem you set out to solve?
- Is it efficient? Does it use resources reasonably?
- Is it elegant? Is it simple yet effective?
- Is it usable? Does it provide a satisfactory way for the target audience to use it?

This chapter explores how to answer these questions for your own solutions.

IS IT CORRECT?

The most important question to ask of your solution is: does it actually solve the original problem you set out to solve? In other words, is it a correct solution? If it's not, then that basically renders every other measure of quality redundant. It doesn't matter how fast, clever or sexy your solution is; if it doesn't solve your problem, then it's incorrect and you need to make changes.

The prudent approach is to assume a program is incorrect until shown to be correct. Practically speaking, this makes sense; after all, a solution is rarely correct before testing and debugging has been done. Philosophically, it's also the right thing to do. Endless possible incorrect solutions exist; there are comparatively few correct ones. It's the same sceptical approach used in fields like science, medicine and law, where the burden of proof lies with those making positive claims, so as to guard against false positives.

Technically speaking, program correctness is a deeply theoretical issue in computer science. In academic circles, talk of proving a program to be correct usually refers to **mathematical** proof. Such a proof, if successful, establishes a program's correctness as incontrovertible fact. However, practitioners outside academia rarely invoke this costly approach when not dealing with critical systems.[40]

Most of the time, showing a program to be correct is done **empirically** by carrying out a series of tests, something covered in Chapter 5. At the beginning of a development process, a problem is specified a certain way. This specification is mirrored in the test plan (see an example plan in Figure 6.1), which rephrases each requirement as a test. The testing runs each aspect of the solution and verifies that the resultant behaviour fulfils the specification.

Figure 6.1 Excerpt from a test plan for a vending machine

Test No.	Test subject	Test description	Expected outcome	Actual outcome
1	Coin slot	Insert an acceptable 50 cent coin	Coin accepted. '0.50' appears on display.	Behaved as expected.
2	Coin slot	Insert an unacceptable foreign coin.	Coin rejected. 'Unrecognised currency' appears on display.	Failed. Coin was rejected but no message appeared.
3	Keypad	Enter valid selection '15'.	'You selected hot chocolate' appears on display.	Behaved as expected.
4	Keypad	Enter invalid selection '99'.	'Invalid selection, please try again' appears on display.	Behaved as expected.

As opposed to the mathematical approach, which aims to prove a solution correct, the aim of an empirical testing approach is to show that the solution is incorrect. This is easier because just one failed test is enough to demonstrate that. It's usually infeasible to test every possible value that could be input to a solution (and every combination thereof), so every test that passes adds one more piece of supporting evidence.

More supporting evidence further strengthens your claims of the solution's correctness. Consequently, showing correctness via testing is not a binary matter of true or false. Rather, testing gives a gradated picture of correctness. This set of levels allows you to grade the correctness of your solution, from least to most convincing:

1. The solution contains no syntax errors or invalid operations that can be automatically detected.
2. The solution yields the correct answer for some set of test data.
3. The solution yields the correct answer for test data that are typical or random.
4. The solution yields the correct answer for test data that are deliberately difficult to process.
5. The solution yields the correct answer for all test data that are possible with respect to the original problem specification.
6. Same as level 5, except a correct (or reasonable) response are given for erroneous test data.
7. For all possible input, the solution gives correct or reasonable responses.

Reducing the number of failing tests doesn't strengthen your claim to a solution's correctness. The presence of even one failing test shows your solution to be incorrect. In other words:

- No passing tests and no failing tests means you have no evidence of correctness.
- Some passing tests and no failing tests means you have some evidence of a correct solution.
- Some passing tests and at least one failing test means you have an incorrect solution.

Correctness shows only that the solution solves the problem. It doesn't matter if it's slow, inefficient, too complicated or someone else simply doesn't like the solution, this doesn't affect correctness.

IS IT EFFICIENT?

Every algorithm requires some amount of resources to do its work. Different algorithms, even ones that solve the same problem, can perform differently in terms of efficiency. This performance is usually measured in terms of time and space.

- **Time:** the duration of an algorithm's running time, from start to end. The duration can be measured as the number of steps taken during execution.
- **Space:** the amount of memory storage required by an algorithm to do its work.

The question of an algorithm's efficiency comes down to its time and space requirements. If, when you analyse an algorithm you find that its use of resources is acceptable, then it may be deemed efficient.

But can its efficiency really be measured reliably in a way that can be fairly compared to that of other algorithms? At first glance it may seem not to be the case. After all, algorithms run on input data and they perform differently depending on the size of input data they get. Imagine an algorithm that totals up a list of numbers: its processing time increases the more numbers are in the list.

Complexity class: a profile that tells you how algorithms belonging to it will behave in general.

This is all true, but CS can measure algorithm efficiency because it examines computation **in general**. It provides general principles about efficiency that you can apply to your specific situation. CS doesn't rank every conceivable, possible algorithm in terms of efficiency.[41] Instead, it provides a relatively small set of complexity classes, into which every algorithm can be placed. It even takes into account that input data size can vary.

Judge your algorithm's efficiency by analysing its behaviour and matching it to one of those categories.

Each category describes how its algorithms perform **in the worst case**. In other words, what the maximum amount of resources an algorithm might use is. Because an algorithm's performance depends on the input data, the resource usage is a function of input data size. Table 6.1 shows some example categories and their efficiency in terms of time.

Table 6.1 Sample of common complexity classes

Name	Running time	Comments
Constant	O(1)	Doesn't necessarily take 1 time unit, rather it takes a fixed number of time units. An example is determining if a number is odd or even.
Logarithmic	O(*logN*)	As the input size grows, the running time increases at an ever-smaller rate. Divide-and-conquer approaches often perform like this.
Linear	O(*N*)	This is the best possible performance of any algorithm that must go through the entire input sequentially.
Quadratic	O(N^2)	The running time of algorithms in this class quadruples when the input size doubles.
Factorial	O(*N*!)	Algorithms in this class are often brute force and considered too slow for practical use.

In Table 6.1, the running time of each category is expressed in a funny looking notation imaginatively called 'Big-O', thanks to its use of a big letter O. The term inside the parentheses denotes how much time algorithms of this class generally require to process input. N stands for the number of items contained in the input.

Here are some rough examples. An algorithm that carries out one step (or one set of steps) for each item is classed $O(N)$. You can say it takes 1 time unit to process input with just 1 item, or it takes 5 time units to process 5 items. In more general terms, it takes N time units to process N items. As another example, an algorithm that carries out N steps for each item,[42] that is $N \times N$, is classified as $O(N^2)$. It takes 4 time units to process 2 items, or 16 time units to process 4 items. More generally, it takes N^2 time units to process N items.

A more concrete example will explain more about complexity classes.

Example: Linear search

A linear search algorithm takes two input parameters: a list of items of size N, and one additional item we'll call a search item (the items could be numbers, pieces of text or whatever). Its goal is to ascertain whether the list contains the search item or not. The algorithm works by searching through the list from start to end, comparing the search item with each item in the list as it goes. As soon as a list item and the search item are found to be equal, the algorithm returns 'true' and immediately halts. If the algorithm reaches the end of the list without finding a match, it returns 'false'. The question is, which complexity category does the algorithm belong to?

- In the best case, the search item matches with the first item in the list. This is $O(1)$.
- On average (assuming each item is just as likely to be sought as any other), the search item will match somewhere in the middle on the list, meaning its average performance is $O\left(\frac{N}{2}\right)$
- In the worst case, either the search item matches with the last item in the list or it isn't found at all. That gives a performance of $O(N)$.

Someone wanting to use a linear search algorithm now has some knowledge to apply. They should have an idea of what kinds of list sizes their solution will have to process, as well as how long it takes to carry out one comparison. Knowing that their chosen algorithm has a worst-case performance of $O(N)$, they can decide whether this is acceptable for their solution.

As you can see, not all algorithms are created equal in terms of performance. Algorithms with worst-case performances like $O(logN)$, $O(N)$ and $O(N^2)$ are considered efficient because their running times are usually acceptable even when N is very large. Algorithms with worst-case performances like $O(2^N)$, $O(N^N)$ or $O(N!)$ are generally considered inefficient because their running times become unacceptable relatively quickly as N grows larger.

Big-O Cheat Sheet (bigocheatsheet.com) categorises many popular data structures and algorithms.

IS IT ELEGANT?

So far, we've seen that evaluation is an objective endeavour. A software solution is not a piece of art;[43] it's a functional thing, just like a bridge, a scientific theory or a mathematical proof. As such, its correctness and efficiency don't come down to a matter of opinion.

However, some aspects of evaluation cause things to become a little fuzzier in this regard. One of these aspects is elegance, something you might associate more with artistic pursuits. Two different solutions might both solve a problem, but they could be judged apart by the elegance of their respective approaches. This applies not only to software; it's true of other 'functional' disciplines like engineering, science and mathematics.

Roughly speaking, elegance maximises both effectiveness and simplicity at the same time.

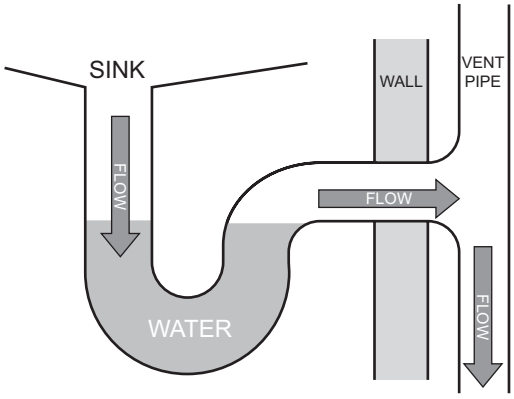
To an engineer, something achieves elegance if it performs something very useful with only a few, simple parts. It might also be non-obvious and solve several problems at once. As an example, consider the pipe that drains water away from the sink in your kitchen or bathroom. It's connected to a sewage outlet and so risks leaking foul stenches back up the pipe and into the room. We could prevent this in any number of ways. One would be to attach some kind of pleasant-smelling filter inside the pipe, but that is easily clogged and would require regular cleaning and changing. Or perhaps a check valve that only opens in one direction to allow water to run out? Not bad, but it would require hinges which are at risk of rust. The actual solution is a trap[44]. This allows water to gather (as a side effect of normal usage), which then blocks any air coming back up from the sewage outlet. It's an incredibly simple, non-obvious solution that works automatically and requires little or no maintenance (see Figure 6.2).

To a scientist, an elegant theory explains a lot with very little. Theories like Newton's second law ($F = ma$) or Einstein's mass–energy equivalence ($E = mc^2$) boil hugely complex phenomena down to very simple equations. The same equations can be applied to something as small as atoms or as large as a galaxy. They also serve as the basis for innumerable other theories.

A mathematical proof is elegant if it's a simple, effective solution to a non-trivial problem. An example of mathematical elegance comes from a (possibly apocryphal) story featuring mathematician Carl Friedrich Gauss (Haynes, 2006). One day in his youth, his mathematics teacher gave the students in his class a problem: calculate the sum of the integers from 1 to 100.[45] All the students embarked on the obvious solution; they

started a running total and began adding numbers to it one at a time (1 + 2 = 3, 3 + 3 = 6, 6 + 4 = 10 and so on). All students except Gauss that is. He noticed a pattern in the problem:[46] 1 + 100 = 101, 2 + 99 = 101, 3 + 98 = 101 and so on, right up to 50 + 51 = 101. This pattern showed that those 100 numbers could be grouped into 50 pairs, each of which totalled 101. Instead of carrying out 100 sums, Gauss needed only to carry out a single multiplication (50 × 101) to get the same answer.

Figure 6.2 Cross section of a trap



It's the mathematician's idea of elegance that is perhaps closest to a computer scientist's. If a programmer were to encode these two solutions for a computer, they would look something like this.

The obvious solution:

- Let total = 0;
- For each number, *n*, between 1 and 100:
  - Let total = total + *n*.

Gauss's more elegant solution:

- Let total = 50 x 101.

Gauss's solution achieves the same as the obvious solution, but does so using less space (one variable instead of two) and less time (one step instead of roughly 100). In terms of complexity (see above), if you were to write generic versions of both algorithms to work out the sum of the first *N* integers, the obvious solution would be O(*N*) while Gauss's solution would be the much more efficient O(1).

> Simple can be harder than complex: You have to work hard to get your thinking clean to make it simple.
>
> (Jobs, 1998)

It might seem paradoxical, but it's harder to be simple than complicated. Coming up with elegant solutions takes time and effort. It requires you to get to the essence of the problem and see the patterns within it. It also requires you to apply some out-of-the-box thinking. But it is potentially very rewarding effort, because the pay-offs of an elegant result can be huge.

The main drawback of a non-obvious solution is just that: it's non-obvious. If someone else eventually comes to work on your solution, they need to know how it works. Anyone looking at the obvious solution above can easily figure out it's totalling the numbers 1 to 100. However, looking at Gauss's elegant solution in isolation, it isn't immediately clear what it's trying to achieve. The author may understand it, but what about everyone else? Therefore, you should always make sure non-obvious parts of your solution are accompanied by clear explanations.

When I talk about other people having to understand your solution, that includes you too! You may understand the complexities today, but will you still understand them after leaving your solution and resuming work on it a few weeks or months later? There's a very good chance you will have forgotten how it works. Do yourself a favour: always write clear explanations of how your solution works, **especially** the complex bits.

When coming up with elegant solutions, be on your guard against cleverness for its own sake. Always be ready to justify why you're not taking the obvious approach.[47] If there are no tangible benefits to your non-obvious way, you could be making the solution more complex for no good reason.

IS IT USABLE?

Ultimately, the users of your solution will be people. Your solution must therefore cater to humans and their particular ways. In other words, it must **usable** and ought to give users a positive experience.

Criteria like correctness, efficiency and elegance describe a solution on its own terms. However, usability measures how well something can be used by people in order to achieve their goals. Humans bring additional baggage. We're not machines; we have desires and emotions. We make mistakes and have limited patience. People want solutions that are easy to learn, uncomplicated to use, forgiving of our fallibility, and the less effort required from us the better.

Usability captures these desires. It formalises them into several components that measure how easy and satisfying it is for people to use a solution (Nielsen, 2012). Thankfully,

despite involving humans in the equation, these components are somewhat measurable. They are:

- **Learnability:** how easy is it for users to accomplish basic tasks the first time they encounter the solution?
- **Efficiency:** once users have learned the solution, how quickly can they perform tasks?
- **Memorability:** when users return to the solution after a period of not using it, how easily can they re-establish proficiency?
- **Errors:** how many errors do users make, how severe are these errors and how easily can they recover from the errors?
- **Satisfaction:** how pleasant is it to use the design?

Usability heuristics are general principles you can apply when designing a solution. If you apply them carefully, then you will have less reworking to do at the evaluation stage. Jakob Nielsen (1995), a leading expert on usability, lists ten such heuristics here: www.nngroup.com/articles/ten-usability-heuristics/

Evaluating usability is ultimately about the reactions of real people. In the end, you will have to involve people in the evaluation. This is done in part by an observational study. People representative of your audience should be recruited as test subjects, who then use the solution to carry out a set of pre-defined tasks (see Table 6.2). While the subject is busy, the evaluator observes their work, noting where the subject is successful and where they have difficulties.

Table 6.2 Sample of a list of usability tasks

Task No.	Description
1	Log into the system.
2	Add an email address to your user profile.
3	Buy a cookery book for under £10.
4	Show a list of your past purchases.

Encourage the test subject to be very vocal when they're testing the solution. They should think aloud, expressing what they're trying to do and what their current thoughts are. This is all information for your evaluation. The evaluator may speak during a test (for example, to clarify what the subject said), but they should keep their interventions to a minimum and should not give the subject explicit hints on how to use the solution.

After the subject completes the test, the evaluator provides them with a questionnaire. The questions are typically broad and don't focus on specific features. Instead they ask the subject to rate various factors on a scale. Here's an example set:

Rate the following on a scale of 1 (strongly disagree) to 5 (strongly agree):

1. Overall, I enjoyed using this system.
2. I found this system easy to use.
3. It took me too long to get used to this system.
4. I had problems finding the functions in this system.
5. There were too many steps to carry out the functions in this system.
6. The system was forgiving when I made a mistake.

To gain more insight, you should follow up on the responses. Either allow the user to elaborate on each answer in a free-text format, or interview them and ask for the reasoning behind their responses.

Usability testing doesn't necessarily have to be done after the solution is completed. You could create a mock-up of your system as a paper prototype and then let the test subjects use that to give you feedback. The usual rule applies: the earlier you find problems, the easier it is to fix them.

TRADE-OFFS

Various factors conspire to ensure that no solution turns out perfect. Some of those factors are down to mistakes being made or a shortage of hours in the day.

However, another reason is that you can't always optimise every aspect of a solution at once. Improving a solution in one respect can often compromise it in others. Consider solution efficiency. It is expressed in terms of the time the solution takes and the storage space it requires. Although not always the case, sometimes a solution forces you to trade better time performance in return for increased space usage – or, conversely, accept a slower solution as the cost for decreased space requirements. This is called a space–time trade-off.

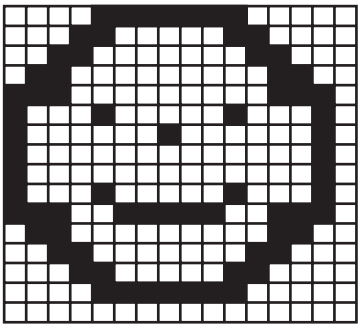
As an illustrative example, let's look at image compression. Images are stored, sent and rendered by computers constantly every day, on disks, in web pages, via video streaming or wherever. For computers to handle an image, it must be broken down into discrete pieces called **pixels** (picture elements), each of which describes one tiny piece of the image.

Storing all the pixels can add up to a lot of information. A typical 3-megapixel camera (which, even at the time of writing, is pretty run-of-the-mill) produces images with over 3 million pixels.[48] Each pixel needs storing in computer memory. In a simple scheme (which we'll use in this example), the image data is stored as a string of tokens. Each token is stored in 1 byte, the smallest addressable size of memory space. A byte

can store a single character or a number between 0 and 255. The colour of a pixel is described by a character token: 'B' means black, 'W' means white.

Consider the 16 × 16 pixel image[49] of a smiley face in Figure 6.3, which has a grid superimposed over it to show the pixels explicitly:

Figure 6.3 Smiley face image

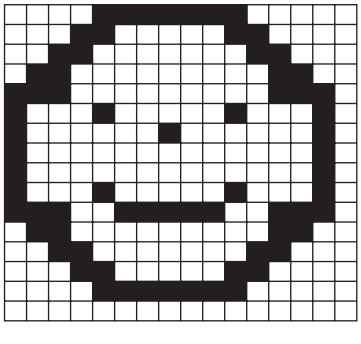


The question here is how to store the image data so the image can be rendered again later. One solution is to store the pixel information in a string of 256 (that is, 16 × 16) tokens, one describing each pixel. Those numbers could be stored on disk or beamed out over the internet for later usage. In this case, rendering means putting the right values (B or W) in the right place (see Figure 6.4).

This would be very fast. To render, the computer needs only go through every pixel from start to finish, grabbing the next value from the string and putting the corresponding pixel in place on the screen. However, this approach is not optimised for space. Notice that most of the image is taken up by a white background. Consequently, the image data contains lots of repeated, contiguous values.

Figure 6.4 Smiley face image with pixel values

Image

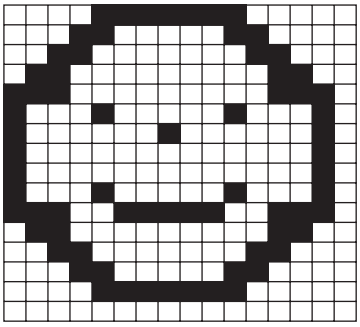


Pixel values

W	W	W	W	B	B	B	B	B	B	B	W	W	W	W	W
W	W	W	B	B	W	W	W	W	W	B	B	W	W	W	W
W	W	B	B	W	W	W	W	W	W	W	B	B	W	W	W
W	B	B	W	W	W	W	W	W	W	W	W	B	B	W	W
B	B	W	W	W	W	W	W	W	W	W	W	W	B	B	W
B	W	W	W	B	W	W	W	W	W	B	W	W	W	B	W
B	W	W	W	W	W	W	B	W	W	W	W	W	W	B	W
B	W	W	W	W	W	W	W	W	W	W	W	W	W	B	W
B	W	W	W	W	W	W	W	W	W	W	W	W	W	B	W
B	W	W	W	B	W	W	W	W	W	B	W	W	W	B	W
B	B	W	W	W	B	B	B	B	B	W	B	B	B	B	W
W	B	B	W	W	W	W	W	W	W	W	W	B	B	W	W
W	W	B	B	W	W	W	W	W	W	W	B	B	W	W	W
W	W	W	W	B	B	W	W	W	W	B	B	W	W	W	W
W	W	W	W	W	B	B	B	B	B	B	W	W	W	W	W
W	W	W	W	W	W	W	W	W	W	W	W	W	W	W	W

Figure 6.5 Smiley face image with compressed pixel values

Image



Pixel values (compressed)

4	W	7	B	5	W										
3	W	2	B	5	W	2	B	4	W						
2	W	2	B	7	W	2	B	3	W						
1	W	2	B	9	W	2	B	2	W						
2	B	11	W	2	B	1	W								
1	B	3	W	1	B	5	W	1	B	3	W	1	B	1	W
1	B	6	W	1	B	6	W	1	B	1	W				
1	B	14	W	1	B	1	W								
1	B	14	W	1	B	1	W								
1	B	3	W	1	B	5	W	1	B	3	W	1	B	1	W
2	B	3	W	5	B	1	W	4	B	1	W				
1	W	2	B	9	W	2	B	2	W						
2	W	2	B	7	W	2	B	2	W						
4	W	2	B	4	W	2	B	4	W						
5	W	6	B	5	W										
16	W														

When data contains a lot of repetition, it can be compressed to take up less space.

We can optimise the amount of space required to store the image by altering the scheme slightly. Instead of describing each pixel, we describe the **patterns** of pixels. For example, when there are five black pixels in a row, instead of storing that information as 'BBBBB' (which requires 5 bytes), we store it as '5B' (which requires 2 bytes: one for the number and one for the pixel colour – see Figure 6.5). By using this scheme, we reduce space requirements. This helps particularly in a situation like sending images over the internet, where reducing the amount of data to be transmitted is greatly beneficial.

Being 16 x 16 pixels, the image data is always stored using 256 bytes when uncompressed. However, using this compression scheme, we can store the smiley face using just 154 bytes. That means the compressed image data is reduced to about 60 per cent of its original size.

However, there's a cost that balances out this saving of space. The amount of work needed to interpret the uncompressed data is less than that of the compressed data. An algorithm for interpreting the uncompressed data may be something like this:

```
p = 0
for each token, t, in the image data:
    make pixel p the colour t
    p = p + 1
```

Whereas interpreting the compressed image data would need an algorithm more like this:

```
p = 0
n = 0
for each token, t, in the image data:
    if t is a number then
        n = t
    else
        repeat n times:
            make pixel p the colour t
            p = p + 1
```

The process of interpreting compressed image data is more complicated and takes more steps in total, and more steps means more time. Thus, we've improved space usage, but at the expense of time usage when rendering.

You might have spotted other examples of trade-offs in earlier sections of this book. The trade-off between cleverness and comprehensibility for example, where incorporating powerful tricks into your solution can make it harder to understand how it works. Or the compromise between usability and flexibility, where solutions that are made more flexible become less user-friendly.[50]

In summary, your solutions will inevitably involve compromises. A situation will often arise along the lines of 'I want better X, but that will make Y worse.' The question then is: how much of Y should you trade-off for X?

Make sure you know beforehand where it makes sense to optimise for your current solution, otherwise you'll have no guiding principles for what trade-offs to make. For example, if your solution involves sending data over a network, then it usually makes sense to optimise for storage space at the expense of additional compression/decompression time. That's because it takes longer to transmit a piece of data than to decode it.

SUMMARY

Evaluation of a solution encompasses many aspects. The most important aspect of a solution is its correctness, that is, does it actually solve the original problem? Beyond correctness, there are numerous other aspects to the quality of a solution. Which ones are important vary from solution to solution. This chapter examined several.

Efficiency measures the time and space usage of a solution to ensure that it solves a problem with acceptable usage of resources. Elegance determines whether a solution maximises both effectiveness and simplicity. Usability introduces a human aspect to evaluation by examining whether the user finds a solution easy and satisfying to use.

In these aspects and more, a degree of trading off is involved. A solution cannot optimise all quality aspects all the time. An essential part of forming a solution is deciding which quality aspects are more important and so which ones are chosen for optimisation.

EXERCISES

EXERCISE 1

Mark the following statements as true or false:

A. Most real-world software solutions are shown be correct empirically rather than mathematically.
B. Reducing the number of failing tests doesn't strengthen your claim to a solution's correctness.
C. An inefficient solution cannot be considered correct.
D. An elegant solution maximises both effectiveness and simplicity at the same time.
E. A usability evaluation begins by explaining to the test subject how to use the system.

EXERCISE 2

Look back at the two examples in the section 'Is it elegant?' Write generic versions of both the obvious solution and Gauss's solution, both of which sum up numbers from 1 to *N*.

EXERCISE 3

Look at the answer for the previous question. Which one of these two algorithms is more efficient in terms of time complexity?

EXERCISE 4

Look back at the image compression example in the section 'Trade-offs'. The compressed image data is not as efficient as it could be. Can you see how the data could be made to take up less space?

Hint: try writing out the data in one long line instead of splitting it over several lines.

EXERCISE 5

What are the names of the five components of usability and what do they measure?

PART II
COMPUTATIONAL THINKING IN SOFTWARE DEVELOPMENT

The second part of this book takes the theoretical computational thinking concepts explained in Part I, and walks you through applying them practically in software programming.

Things you see in Part II can be traced directly back to specific concepts in Part I. Notifications like this point out where in Part I to find them.

These practical lessons are applicable to a wide range of different programming languages.[51] However, for the sake of consistency, a single language will be used for demonstrations of executable source code. Python has been chosen because it is a clean, minimalist language that allows you to build simple, understandable programs quickly.

Keep in mind that Part II is not a comprehensive manual on Python or software programming in general. It only relates the specific principles of computational thinking to relevant aspects of programming.